CTCPU

CTCPU, sayıları iki gösterimle işlemek için tasarladığım bir işlemci. Sayının normal basamaklı yazımını ve gölge dediğim ikinci gösterimini kullanıyor. Hunter bu çalışmanın devamı.

Renklerle başla

Normal bir işlemci bitlerle çalışır: 0 ya da 1. CTCPU'da en küçük birim daha büyük: xbit, yani üç bit, yani sekiz değer. Bu sekiz değeri sekiz renkle gösteriyorum, çünkü üç ışık kanalı (kırmızı, yeşil, mavi) karışınca sekiz renk verir. Her rengin bir numarası ve bir harfi var.

Sayı	Renk	Harf	Üç bit
0	Siyah	K	000
1	Kırmızı	R	001
2	Yeşil	G	010
3	Sarı	Y	011
4	Mavi	B	100
5	Macenta	M	101
6	Camgöbeği	C	110
7	Beyaz	W	111

Renkler ışık gibi karışır: kırmızı (1) + yeşil (2) = sarı (3), kırmızı (1) + mavi (4) = macenta (5), yeşil (2) + mavi (4) = camgöbeği (6), kırmızı + yeşil + mavi = beyaz (7).

Bir sayı, bu renklerden oluşan bir dizidir. 137 sekizlik tabanda 211 yazılır, renkle G R R olur. Mersenne sayısı 31 sekizlikte 37, renkle Y W. 127 sekizlikte 177, renkle R W W. Günlükteki 1, 3, 7 örüntülerini renk olarak görebilmemin sebebi bu.

Kurmak istediğim şey

Amacım sakinliğiyle kazanan bir makine. Bir sayının yapısı ihtiyacım olanı zaten söylüyorsa o zor sayıyı açmaktan kaçınmak istiyorum. Onu ana bilgisayarın yanında, kesin aritmetik ve yapılı sayılarla çalışan bir uzman olarak düşünüyorum. Emülatörümün insanların zaten sahip olduğu makinelerde çalışmasını istiyorum.

Bu bir amaç, ölçülmüş hız veya enerji üstünlüğü değil. Sayıyı, gösterimini ve gösterimler arasında geçmenin bedelini birlikte düşünüyorum. Pascal örüntüleri ile Mersenne sayıları beni bu soruların birçoğuna götürdü. İşe yarayan bir gösterim, bölünebilirlik eleği ve asallık kanıtı yine ayrı şeyler.

Sayıyı tutmanın iki yolu

Normal taraf, `rm`, sekiz sekizlik hane tutuyor, yani 24 bit. İşaretsiz büyüklük olarak 0 ile 16.777.215 arası. Renk, hanenin etiketi. Üç bitin ötesinde bilgi eklemiyor.

Gölge tarafında 2, 3, 5, 7, 11, 13 ve 17 sabit asallarını kullanıyorum. 360 için üsler `[3,2,1,0,0,0,0]`, çünkü `360=2^3×3^2×5`. Toplama normal tarafta kalıyor. Bu yedi asal dışında asal çarpanı olmayan smooth çarpanları çarpmak, üslerini toplamaya dönüşebiliyor.

Gölgenin bütünü bir çarpan listesinden fazlası. Artık kaldığında son sekizlik hanesini tutup geri kalanını aşağı indiriyorum. 59 için ilk satırın asal çarpımı 1, sonraki satırınki 7, saklanan hane 3:

```
59 = 1 × (8 × 7 + 3)
```

Her iniş 8'e bölme. Saklanan hane indiğim satırda duruyor. Eleği, üs sayaçlarını ve merdiveni Resolver sayfasında anlatıyorum. Bütün üsleri sıfır olan satırın değeri 1. Boş satır değil.

Tür ve ölçek de taşıyorum. Kök, ters, toplam veya faktöriyel, açmam gerekene kadar türü olan bir ifade olarak kalabiliyor. Etiketi tutmak açılımı hesaplamıyor. ISA sayfasında sekiz türü ve uygulamalarının sınırlarını veriyorum.

Parçaların bütünü

Güncel Hunter düzeninde tek çekirdek, tek thread ve sekiz register var, `R0..R7`. Her register sekizer hücreli sekiz satır içeriyor. Hücre üç bit, tamamı 192 bit:

```
rm          normal sayı
rs1..rs4    gölge satırları
rc          rough önbellek
rpow        ölçek beyanı
rtyp        sayı türü ve hedeflenen durum alanı
```

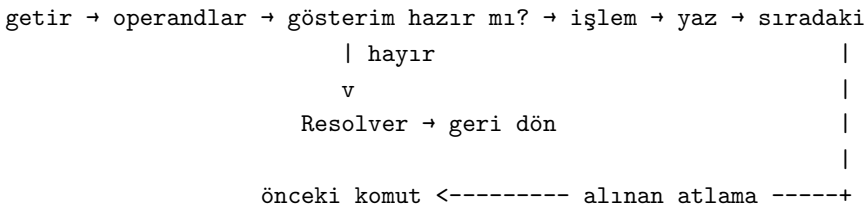
`rs1` bir register'ın içindeki satır, `R1` register'ı değil. `rc` rough önbellek. Toplamadaki elde haneler arasında ilerliyor ve başka bir nesne.

Hücreyi ve register'ları, ALU'yu, toplayıcıyı, MUL_DIV, SQRT, Resolver ve kontrol işlevlerini durumlarıyla CTCPU parçalarında listeliyorum. Eski PCCPU, taşma zinciri ve Küp fikirlerini de tarihsel durumlarıyla orada tutuyorum. Önceki 137 bit düzen, ayrı FLAG ve VLINK, bu 192 bit düzene ait değil.

Yürütme Mantığı: bir komutun yolculuğu

Çalışmadan önce program, giriş değerleri ve başlangıç komut adresi olmalı. Program sayacı sıradaki komutu gösteriyor. Kontrol onun işlemini, değiştiricilerini ve operand seçicilerini okuyor. Hunter'ın kesin adres ve zamanlama arayüzü hâlâ açık. Ama iş bu veri bağımlılığıyla ilerlemek zorunda:

STEP	Önceki durum	İş	Sonraki durum ve karar
1	Program sayacı bir komutu gösteriyor	Dört xbitlik kelimeyi getir	Komut kontrolün önünde
2	OP, MOD, A ve B hazır	İşlemi ve kaynak register'ları seç	Operand değerleri hazır
3	İşlem seçili	Gölge gerekip gerekmediğini belirle	Normal yol devam eder, eksik gölge Resolver ister
4	Gerekiyorsa gölge istendi	Haneleri akıt, üsleri say, artığı işle	Gereken gösterim hazır olunca devam et
5	Operandlar hazır	Seçilen aritmetiği yürüt	Sonuç ve ilgili durumlar hazır
6	Sonuç hazır	Hedefi yaz, gösterim geçerliliğini koru	Yeni hedef değeri kaydedildi
7	Komut bitti	Sıradan devamı veya atlamayı seç	Program sayacı sıradaki komutu gösteriyor



ADD için normal basamakları okuyorum. Gölge çarpmasında gölge eksikse önce Resolver tetikleniyor. Basit bir smooth çarpma üsleri toplayabiliyor. Ama genel rough ve merdiven işi atlanamaz. Normal değeri yazarken eski gölgeyi de güncellemek veya geçersiz saymak gerekiyor.

Döngüyü programın koşulu veya sayacı bitiriyor. Sistem durma isteğinin yürütmeyi durdurması hedefleniyor. Ama Hunter SYS alt kodu ve atlama hedefinin kodlaması açık. Bu yedi anlatım adımına sabit bir fiziksel saat çevrimi sayısı vermiyorum.

Küçük bir örnek: 6 × 7

Tam bir küçük döngüyü göstermek için ayrı HunterCPU öğretim modelini kullanıyorum. Onun kendi 8 bit register'ları ve komut kümesi var. Bu tekrarlı toplama. Hunter'ın MUL ailesinin veya modern bir işlemcinin MUL komutunun böyle çalışmak zorunda olduğu iddiası değil.

`R2` eklenecek 7'yi tutuyor. `R1` biriken sonucu tutuyor. `R0` hâlâ eklenmesi gereken kopyaları sayıyor. Model R0=6 ve R2=7 yüklüyor, azaltmadan önce sıfır durumunu kontrol ediyor, ilk 7'yi doğrudan RAM'den R1'e yüklüyor, sonra R0'ı 5'e indiriyor. Bir kopya zaten var, beş ekleme kaldı.

```
R1=7, R2=7, R0=5
        |
        v
   R0 sıfır mı? -- evet ---> R1'i sakla → dur
        | hayır
        v
   R1 = R1 + R2
        |
   elde var mı? -- evet ---> taşma çıkışı
        | hayır
        v
   R0 = R0 - 1
        |
        +-----------------> sıfır kontrolüne dön
```

STEP	R1 önce	R0 önce	İş	R1 sonra	R0 sonra	Sıradaki karar
Başlangıç	0	6	İlk 7'yi yükle, bir kopyayı çıkar	7	5	Devam
1	7	5	7 ekle, eldeyi denetle, azalt	14	4	Devam
2	14	4	7 ekle, eldeyi denetle, azalt	21	3	Devam
3	21	3	7 ekle, eldeyi denetle, azalt	28	2	Devam
4	28	2	7 ekle, eldeyi denetle, azalt	35	1	Devam
5	35	1	7 ekle, eldeyi denetle, azalt	42	0	Sonraki sıfır kontrolünde bitir
Bitiş	42	0	Sonucu sakla, dur	42	0	Durdu

Bu modelde OR, R0'ı kontrol edip Z'yi kuruyor. JZ, Z'yi okuyor, 1 olduğunda bitiş dalını alıyor. ADD yeni toplamı ve C eldesini üretiyor. JC taşma çıkışı için C'yi okuyor. Elde yoksa sayaç azalıyor, JMP döngü başına dönüyor. Her döngü girişinde `R1 + R0×R2 = 42` eşitliği korunuyor. Tamamlanan her tur, kalan kopyalardan bir 7'yi R1'e geçiriyor. R0 sıfıra geldiğinde R1'in 42 olması gerekiyor.

2 Ekim 2026 raporunda bu öğretim programı için şu sayımlar var:

Sayılan olay	Adet
7'nin mantıksal kopyası	6
Sonuç toplaması	5
Sonuç ADD kaynağı olarak R2 okuması	5
Sayaç azaltma	6, döngüden önce bir ve içinde beş
Döngü sıfır kontrolü / JZ kararı	6, beş devam ve bir bitiş
Ayrı ilk sıfır kontrolü	1
Elde kontrolü / JC kararı	5
Döngüye dönen JMP	5
Sayaç işleri dahil bütün ADD komutları	11
HLT dahil tamamlanan komut	45
Model tiki	268
Bellek okuma / yazma	59 / 1

R2 bir kez yükleniyor, ardından eklenen sayı olarak beş kez okunuyor. R1'deki ilk kopya RAM'den geliyor, ek bir R2 okumasından değil. Bu kaynak okuma sayıları mikro adımların bütün iç aktarımlarını saymıyor. Modelin 268 tiki kaydedilmiş emülatör sonucu, fiziksel Hunter zamanlama ölçümü değil. Bu raporu burada yeniden çalıştırmadım.

Bir toplama içinde eski R1, BUS üzerinden TMP'ye gidiyor. ALU, R2 ile TMP'yi birleştirip sonucu ACC'ye koyuyor. Sonraki aktarım ACC'yi BUS üzerinden R1'e götürüyor. Ekrandaki geçici mor "gölge", bu ACC aşaması. Asal üs gölgesi `rs1..rs4` değil. R0 daha sonra, elde kontrolünden sonra azalıyor. Kaydedilmiş koşuda 42, 222. tikte R1'e ulaşıyor. Program 268. tikte duruyor.

Sıfır, `sayaç-1` kurulmadan önce ele alınmalı. Bir kopya için tekrarlı sonuç toplaması gerekmiyor. Negatif girdiler işaret kuralı istiyor. 8 bitlik sonuç 255'i aşarsa taşma yolu seçiliyor. Bu küçük işaretsiz örnek genel bir işaretli çarpma yordamı sağlamıyor.

Görünür tuttuğum sınırlar

Kesin aritmetik hedefim sonlu saklama sınırlarını kaldırmıyor. Normal alan 24 bit, üs hücresi üç bit, gölge dört satır. Tür ve ölçek ayrıntıları bütünüyle kararlaştırılmış değil. Eksik bitiş kuralının yerine kullanılmayan satırı sessizce sıfır veya bir sayamam.

Resolver'ın eski bölünebilirlik kontrolleri tarihsel emülatör sonuçları. Katmanlı rough gösterimi o kodda hâlâ yok. Genel rough aritmetiği, zincirin sonu, `rc` yorumu ve ISA'nın bazı kısımları açık. Rastgele girdilerde işe yarayan bir smooth kısım kendiliğinden oluşmuyor. Daha kısa matematiksel ifadeden hız üstünlüğü çıkaramam.

Günlüğün 22 test kartı bütünleşik CPU'yu değil, günlükteki iddiaları sınıyor. Bu paket 30 arşiv testi ile TESTLER/NIHAI_EK altındaki iki programı içeriyor. Arşivde öğretim modelleri ve başlangıç RTL çalışmaları var. Ama bu yayın kabul edilmiş FPGA uygulaması

veya silikon sonucu sunmuyor. MHz, watt ve fiziksel saat çevrimi sayıları burada ölçülmüş değil. Eski 4096 thread hayali projenin tarihine ait, bugünkü tek çekirdeğin kapasitesine değil.

Parçalar, Resolver, ISA, English

James Augustus, 2026. CC BY 4.0 lisansıyla. https://github.com/JamesAugustus/ctcpu-hunter